

Esame OS Internals (Cabodi)

Data: 02/09/2021

Nota Generale: TUTTE LE RISPOSTE SI/NO DEVONO ESSERE MOTIVATE. QUANDO I RISULTATI SONO NUMERI, SONO RICHIESTI SIA IL RISULTATO FINALE SIA I RELATIVI PASSI INTERMEDI (O FORMULE).

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Second Chance e Codice C)

Contesto: Sia dato il frammento di programma rappresentato:

```
#define N (128*1024)
int i, j;
int A[1024], B[N];

for (j=0; j<N; j++) {
    B[j] = 0;
}

for (i=0; i<1024; i++) {
    j = A[i];
    if (j>=0 && j<N)
        B[j]++;
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con memoria virtuale gestita mediante paginazione, con pagine di 2 KByte, utilizzando come politica di sostituzione pagine la SECOND CHANCE. Si sa che un `int` ha dimensione 32 bit e che le istruzioni in codice macchina, corrispondenti al frammento di programma visualizzato, sono contenute in una sola pagina. Si supponga che `A` e `B` siano allocati ad indirizzi logici contigui (prima `A`, poi `B`), a partire dall'indirizzo logico `0x74240A00`.

Si risponda alle seguenti domande:

- A) Quante pagine (e frame) sono necessarie per contenere i vettori `A` e `B`?
- A) Ipotizzando che le variabili `i`, `j` siano allocate in registri della CPU, quanti accessi in memoria (in lettura e scrittura) fa il programma proposto, per accedere a dati (non vanno conteggiati gli accessi a istruzioni)? (Siccome il costrutto `if (j>=0 && j<N)` condiziona un'istruzione, si calcolino sia il caso minimo che il caso massimo).
- A) Calcolare il numero di page fault generati dal programma proposto, supponendo che siano allocati per esso 10 frame, di cui uno utilizzato (già all'inizio dell'esecuzione) per le istruzioni. (Considerare nuovamente caso minimo e massimo, e motivare le risposte).

Domanda 2 (Memorie di Massa e Dischi SSD)

Contesto: Sia dato un disco di tipo "solid state" (SSD). Una delle tecniche usate per prolungare la vita di un dispositivo SSD è il cosiddetto "wear leveling", che consiste nel mantenere una tabella (mappa) di corrispondenza tra indirizzi logici e fisici per i blocchi su disco. Implementata in varie modalità, tale tecnica cerca di "spostare" un dato blocco logico da un blocco fisico (pagina, nella terminologia SSD) a un altro in corrispondenza a un'operazione di "ri-scrittura".

Si risponda alle seguenti domande:

- A) Perché è opportuna questa operazione?
 - A) Da cosa nasce il possibile vantaggio, da un numero minore complessivo di scritture su disco, oppure da qualcos'altro?
 - A) E' possibile che un disco SSD continui a funzionare correttamente una volta che una o più pagine (blocchi) fisiche siano marcate come inutilizzabili (in quanto hanno raggiunto il numero massimo di riscritture)? (Se sì, spiegare come; se no, motivare).
-

Domanda 3 (I/O Sincrono, Asincrono e Bloccante)

Contesto: Si considerino i due tipi di sincronizzazione, relativi a operazioni di I/O: sincrono e asincrono.

Si risponda alle seguenti domande:

- A) Si spieghino le principali differenze tra gli I/O dei due tipi. Si considerino poi I/O bloccante e non bloccante: si tratta di sinonimi di asincrono e sincrono? Ci sono differenze (tra bloccante/non-bloccante e sincrono/asincrono)? (se no, perché; se sì, quali?).
- A) Un I/O sincrono può essere effettuato in polling oppure è necessario effettuarlo in interrupt? (Si risponda alla stessa domanda per il caso dell'I/O asincrono).
- A) In quale modo può un processo beneficiare di un I/O asincrono? E' possibile, nel caso di I/O asincrono, scrivere un programma con istruzioni, successive all'I/O, che dipendano dai dati coinvolti (ad es. letti) durante l'I/O?

2 SECTION 2: OS161

Domanda 4 (System Call `sys_read` e Memoria)

Contesto: Sia dato un sistema OS161. Si consideri la funzione `sys_read`, utilizzata per realizzare la system call `read`. Se ne propone una realizzazione, limitata al caso di standard input.

```
int sys_read(int fd, userptr_t buf, int size) {
    if (fd == STDIN_FILENO) {
        kgets ((char *)buf, size);
        return size;
    } else {
        kprintf("sys_read implemented only on stdio\n");
        return -1;
    }
}
```

Si risponda alle seguenti domande:

- B) E' possibile che la funzione sia richiamata direttamente da un processo user (ad esempio mediante una chiamata interna a `palin.c`, `sort.c`, `forktest.c`)? (Motivare la risposta).
 - B) La funzione è errata, in quanto la chiamata a `kgets`, così come effettuata, rischia di fare un accesso scorretto in memoria. Si spieghi il perché e si proponga una possibile correzione al problema.
 - B) Perché la funzione ritorna `size`, qualora non ci sia errore? E' possibile che, in una realizzazione più completa della funzione, il valore ritornato sia maggiore o minore di `size`?
-

Domanda 5 (Sincronizzazione Multi-core e Semafori)

Contesto: Sia dato un sistema operativo OS161.

Si risponda alle seguenti domande:

- B) Si spieghi perché, in un contesto multi-core (sono presenti più CPU) non è possibile realizzare la mutua esclusione semplicemente disabilitando e riabilitando l'interrupt.
- B) Dato codice (ridotto alle parti essenziali) delle funzioni di semaforo P e V, riportate in seguito:

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0) {
        wchan_sleep(sem->sem_wchan, &sem->sem_lock);
    }
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}

void V(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
```

```
sem->sem_count++;  
KASSERT(sem->sem_count > 0);  
wchan_wakeone(sem->sem_wchan, &sem->sem_lock);  
spinlock_release(&sem->sem_lock);  
}
```

1. A cosa serve lo spinlock (in entrambe le funzioni)?
2. Perché la P contiene un ciclo `while` anziché un `if (sem->sem_count == 0)?`
3. Perché la `wchan_sleep` riceve come parametro lo spinlock?
4. E' possibile che la chiamata alla `wchan_wakeone` svegli più di un thread in attesa su `wait_channel`?

B) Spiegare brevemente la differenza tra condition variable e `wait_channel`.